

Per avere PDA più efficienti / più piccoli
e con minor nondeterminismo è necessario

(20)

SEMPLIFICARE LE GRAMMATICHE

- 1) Eliminare le produzioni ϵ (del tipo $A \rightarrow \epsilon$)
che sono inadatte al bottom-up parsing
- 2) Eliminare le produzioni unitarie (del tipo $A \rightarrow B$)
che possono creare dei cicli $A \Rightarrow^+ A$
- 3) Eliminare simboli inutili, cioè quei terminali
e nonterminali che non sono raggiungibili/generabili
a partire dal simbolo iniziale S
- 4) Eliminare la ricorsione sinistra (del tipo
 $A \rightarrow A\alpha$), perché inadatta al top-down parsing
- 5) Fattorizzare la grammatica, per ottenere
grammatiche con meno nondeterminismo
nel top-down parsing.

Eliminare le produzioni ϵ

(21)

- Input: G libera con produzioni ϵ (del tipo $A \rightarrow \epsilon$)
- Output: G' libera, senza produzioni ϵ , tale che

$$L(G') = L(G) \setminus \{\epsilon\}$$

Oss: Se $\epsilon \in L(G)$ e vogliamo una G'' tale che $L(G) = L(G'')$, basta considerare $G' = (NT, T, S, R')$ e definire $G'' = G' \cup \{S' \rightarrow \epsilon \mid S'\}$ ↑ output dell'algoritmo

$$G'' = (NT \cup \{S'\}, T, S', R' \cup \{S' \rightarrow \epsilon \mid S'\})$$

Simboli annullabili: $A \in NT$ tale che $A \Rightarrow^+ \epsilon$

$N(G) = \{A \in NT \mid A \Rightarrow^+ \epsilon\}$ viene calcolato induttivamente come segue:

$$N_0(G) = \{A \in NT \mid A \rightarrow \epsilon \in R\}$$

$$N_{i+1}(G) = N_i(G) \cup \{B \in NT \mid B \rightarrow c_1 \dots c_k \in R \text{ e } c_1, \dots, c_k \in N_i(G)\}$$

Oss: • $N_i(G) \subseteq N_{i+1}(G)$ perché aggiungo qualcosa ad ogni passo

• $\exists i_c$ tale che $N_{i_c}(G) = N_{i_c+1}(G)$ perché NT è finito!

Fatto: L'insieme $N(G) = N_{i_c}(G)$ è esattamente l'insieme di tutti i simboli annullabili

Una volta calcolato $N(G)$ per $G=(NT, T, S, R)$, (22)

costruiamo la grammatica $G'=(NT, T, S, R')$ dove per ogni produzione $A \rightarrow \alpha \in R$, con $\alpha \neq \epsilon$, in cui occorrono i simboli annullabili $z_1, \dots, z_k$, mettiamo in R' tutte le produzioni del tipo $A \rightarrow \alpha'$ dove α' si ottiene da α cancellando tutti i possibili sottoinsiemi di $z_1, \dots, z_k$ (incluso $\emptyset$), ad eccezione del caso in cui α' risulta ϵ .

- • in G' non mettiamo produzioni $A \rightarrow \epsilon \in R$
- • in G' non introduciamo mai produzioni del tipo $A \rightarrow \epsilon$

Proposizione Data una grammatica libera G , la grammatica G' determinata dall'algoritmo sopra non ha ϵ -produzioni e $L(G') = L(G) \setminus \{\epsilon\}$

(Come già detto, se vogliamo che la nuova grammatica sia del tutto equivalente a G , allora bisogna ammettere una produzione ϵ per il nuovo simbolo iniziale;

$$G'' = (NT \cup \{S'\}, T, S', R' \cup \{S' \rightarrow \epsilon \mid S\})$$

Esempio:

(23)

$$G = \begin{cases} S \rightarrow AB \\ A \rightarrow aAA \mid \epsilon \\ B \rightarrow bBB \mid \epsilon \end{cases}$$

$$N_0(G) = \{A, B\}$$

$$N_1(G) = \{A, B, S\} \\ = N(G)$$

Consideriamo $S \rightarrow AB \in R$. Dobbiamo considerare 4 casi:

$$\emptyset \Rightarrow S \rightarrow AB$$

$$\{B\} \Rightarrow S \rightarrow A$$

$$\{A\} \Rightarrow S \rightarrow B$$

$$\{A, B\} \Rightarrow S \rightarrow \epsilon \quad \text{non la mettiamo in } R'$$

$$S \rightarrow AB \mid A \mid B$$

$\Rightarrow$

Ora consideriamo $A \rightarrow aAA \in R$. Dobbiamo considerare

4 casi:

$$\emptyset \Rightarrow A \rightarrow aAA$$

$$\{A\} \Rightarrow A \rightarrow aA$$

$$\{A\} \Rightarrow A \rightarrow aA$$

$$\{A, A\} \Rightarrow A \rightarrow a$$

} duplicate

$$\Rightarrow A \rightarrow aAA \mid aA \mid a$$

Ora consideriamo $B \rightarrow bBB$, e nello stesso modo di prima, otteniamo $B \rightarrow bBB \mid bB \mid b$

Le due ϵ -productions $A \rightarrow \epsilon$ e $B \rightarrow \epsilon$ sono ignorate.

$$\Rightarrow G' = \begin{cases} S \rightarrow AB \mid A \mid B \\ A \rightarrow aAA \mid aA \mid a \\ B \rightarrow bBB \mid bB \mid b \end{cases}$$

N.B. $\epsilon \in L(G)$ $S \Rightarrow AB \Rightarrow B \Rightarrow \epsilon$
 $\epsilon \notin L(G')$

$$G'' = \begin{cases} S' \rightarrow \epsilon \mid S \\ S \text{ come per } G' \end{cases} \quad \text{e' tale che } L(G) = L(G'')$$

Eliminare le produzioni unitarie

(24)

- produzioni unitarie: $A \rightarrow B$ con $A, B \in NT$
- coppie unitarie: (A, B) tale che $A \Rightarrow^* B$ usando solo produzioni unitarie

Calcolare le coppie unitarie induttivamente

$$1) U_0(G) = \{ (A, A) \mid A \in NT \}$$

$$2) U_{i+1}(G) = U_i(G) \cup \{ (A, C) \mid (A, B) \in U_i(G) \text{ e } B \rightarrow C \in R \}$$

Oss: $U_i(G) \subseteq U_{i+1}(G)$

• $\exists i_c$ tale che $U_{i_c}(G) = U_{i_c+1}(G)$ perché NT è finito

Per definizione, $U(G) = U_{i_c}(G)$, detto insieme di tutte le coppie unitarie.

Algoritmo: Data $G = (NT, T, R, S)$ libera, si definisce $G' = (NT, T, R', S)$ dove, per ogni $(A, B) \in U(G)$, R' contiene tutte le produzioni $A \rightarrow \alpha$, dove $B \rightarrow \alpha \in R$ e non è unitaria.

Oss: Poiché, per ogni $A \in NT$, la coppia $(A, A) \in U(G)$, R' contiene tutte le produzioni non-unitarie di R e in aggiunta un po' di altre.

Teorema

(25)

Sia $G = (NT, T, R, S)$ libera e sia $U(G)$ l'insieme delle sue coppie unitarie. Sia $G' = (NT, T, R', S)$ la grammatica ottenuta dall'algoritmo sopra. Allora G' non ha produzioni unitarie e $L(G) = L(G')$.

Esempio

$E \rightarrow E + T \mid T$	} G	gram. non ambigua per espr. aritmetiche • però ha prod. unitarie $E \rightarrow T$ e $T \rightarrow A$
$T \rightarrow T * A \mid A$		
$A \rightarrow a \mid b \mid (E)$		

$$U_0(G) = \{(E, E), (T, T), (A, A)\}$$

$$U_1(G) = U_0(G) \cup \{(E, T), (T, A)\}$$

$$U_2(G) = U_1(G) \cup \{(E, A)\} = U_3(G) = U(G)$$

$G' \begin{cases} E \rightarrow E + T \\ T \rightarrow T * A \\ A \rightarrow a \mid b \mid (E) \end{cases}$ produzioni presenti
grazie a $U_0(G)$

in aggiunta

$$E \rightarrow T * A$$

perché $(E, T) \in U_1(G)$

$$T \rightarrow a \mid b \mid (E)$$

perché $(T, A) \in U_1(G)$

$$E \rightarrow a \mid b \mid (E)$$

perché $(E, A) \in U_2(G)$

Quindi, riassumendo, G' è

$$E \rightarrow E + T \mid T * A \mid a \mid b \mid (E)$$

$$T \rightarrow T * A \mid a \mid b \mid (E)$$

$$A \rightarrow a \mid b \mid (E)$$

non contiene produzioni unitarie ed è equivalente
a G

Rimuovere i simboli inutili

(26)

Def: Un simbolo $X \in T \cup N$ è

- un generatore se $\exists w \in T^*$ con $X \Rightarrow^* w$
- raggiungibile se $S \Rightarrow^* \alpha X \beta$ per qualche $\alpha, \beta \in (T \cup N)^*$
- utile se è sia generatore, sia raggiungibile
ovvero se $S \Rightarrow^* \alpha X \beta \Rightarrow^* z \in L(G)$, cioè X
compare in almeno una derivazione di una
stringa $z \in L(G)$.

Esempio

$$\left. \begin{array}{l} S \rightarrow AB \mid a \\ B \rightarrow b \end{array} \right] G$$

Poiché

$$\left. \begin{array}{l} a \Rightarrow^* a \\ b \Rightarrow^* b \\ S \Rightarrow^* a \\ B \Rightarrow^* b \end{array} \right\} \Rightarrow \text{generatori} = \{S, B, a, b\}$$

(manca A)

$\Downarrow$
possiamo eliminare
tutte le produzioni che
includono A

$$G' \left[\begin{array}{l} S \rightarrow a \\ B \rightarrow b \end{array} \right]$$

Ora B non è raggiungibile da S, allora eliminiamo B

$$G'' \left[S \rightarrow a \right]$$

G'' è equivalente a G , ma non contiene
simboli inutili

Come calcolare i generatori?

(27)

Ricorda: X è un generatore se $\exists w \in T^*$. $X \Rightarrow^* w$

1) $G_0(G) = T$

se $a \in T$, $a \Rightarrow^* a$!

allora tutti i terminali
sono generatori

2) $G_{i+1}(G) = G_i(G) \cup$

$$\left\{ B \in NT \mid B \rightarrow C_1 \dots C_k \in R \right. \\ \left. \text{e } C_1, \dots, C_k \in G_i(G) \right\}$$

Oss: Nel passo 2) è contemplato anche il caso
 $k=0$ (cioè $B \rightarrow \varepsilon \in R$): se $B \rightarrow \varepsilon$, allora
 B è un generatore.

Oss: • $G_i(G) \subseteq G_{i+1}(G)$

• $\exists i_c$ tale che $G_{i_c}(G) = G_{i_c+1}(G)$ perché
TUNT è finito

Fatto L'insieme $G(G) = G_{i_c}(G)$ è esattamente
l'insieme di tutti i simboli generatori
di G .

Come calcolare i raggiungibili?

(28)

Ricorda: X è raggiungibile se

$$S \Rightarrow^* \alpha X \beta \quad \text{per qualche } \alpha, \beta \in (NT \cup T)^*$$

$$1) R_0(G) = \{S\}$$

$$2) R_{i+1}(G) = R_i(G) \cup$$

$$\bigcup \{X_1, \dots, X_k\}$$

$$B \in R_i(G),$$

$$B \rightarrow X_1 \dots X_k \in R$$

Oss: $R_i(G) \subseteq R_{i+1}(G)$

$\exists i_c. R_{i_c}(G) = R_{i_c+1}(G)$ perché $T \cup NT$ è finito

Fatto L'insieme $R(G) = R_{i_c}(G)$ è esattamente l'insieme di tutti i simboli raggiungibili di G .

Come rimuovere i simboli inutili?

1) Prima elimino tutti i non-generatori
(e tutte le produzioni che usano almeno uno di questi)

2) Poi, dalla nuova grammatica, elimino tutti i non-raggiungibili (e tutte le prod. che li usano)

$\Rightarrow$ La grammatica risultante è equivalente all'originale e non contiene simboli inutili

Teorema Sia $G = (NT, T, R, S)$ una grammatica libera tale che $L(G) \neq \emptyset$.

- 1) Sia G_1 la grammatica che si ottiene da G eliminando tutti i simboli che non appartengono a $G(G)$, e tutte le produzioni che fanno uso di almeno uno di tali simboli. $\otimes$
- 2) Sia G_2 la grammatica che si ottiene da G_1 eliminando tutti i simboli che non appartengono a $R(G_1)$, e tutte le produzioni che fanno uso di almeno uno di tali simboli.

Allora G_2 non ha simboli inutili e $L(G_2) = L(G)$.

Dimostrazione

- $L(G_2) \subseteq L(G)$ è ovvio, perché G_2 contiene meno produzioni di G .

Poiché $L(G) \neq \emptyset$, allora S è un generatore e così rimane in G_1 , e poi in G_2

- $L(G) \subseteq L(G_2)$: dobbiamo dimostrare

che se $S \Rightarrow_G^* w$ allora $S \Rightarrow_{G_2}^* w$. Ma ogni simbolo usato nella derivazione $S \Rightarrow_G^* w$ è ovviamente sia raggiungibile, sia generatore!! Allora quella derivazione è anche una derivazione per G_2 .

L'ordine dei 2 passi è importante!

(30)

- 1) prima elimino i non-generatori
- 2) poi i non-raggiungibili.

Ma se inverto l'ordine, allora può capitare che non elimino tutti i simboli inutili !!

Esempio

$S \rightarrow AB \mid a$

$B \rightarrow b$

1) Elimino i non-generatori

$S \rightarrow a$

$B \rightarrow b$

2) Elimino i non-raggiungibili

$S \rightarrow a$

Corretto!

tutti i simboli inutili
sono stati rimossi

2) Elimino i non-raggiungibili

$S \rightarrow AB \mid a$

$B \rightarrow b$

Tutti
raff.

1) Elimino i non-generatori

$S \rightarrow a$

$B \rightarrow b$

è rimasto un
simbolo inutile: B
e la prod. $B \rightarrow b$

Esempio

(31)

$$\left. \begin{array}{l} S \rightarrow AB \mid aC \\ A \rightarrow a \\ B \rightarrow bB \\ C \rightarrow b \mid AC \\ D \rightarrow a \mid aS \end{array} \right\} G$$

$$G(G) = \{a, b, A, C, D, S\}$$

solo B non è generatore



$$R(G_1) = \{S, a, C, b, A\}$$

$$G_1 \left[\begin{array}{l} S \rightarrow aC \\ A \rightarrow a \\ C \rightarrow b \mid AC \\ D \rightarrow a \mid aS \end{array} \right.$$

solo D non è raggiungibile



$$\left[\begin{array}{l} S \rightarrow aC \\ A \rightarrow a \\ C \rightarrow b \mid AC \end{array} \right] G_2$$

è la grammatica semplificata, equivalente a G, senza simboli inutili.

$$L(G_2) = \{ab, aab, aaab, \dots\} = a^+b$$

È possibile trovare una grammatica più semplice di G_2 per il lang. a^+b ?

$$S \rightarrow aS \mid ab$$

Se, nel semplificare la grammatica G , seguiamo questo ordine:

- 1) eliminare le ϵ -produzioni
- 2) eliminare le produzioni unitarie (e quindi i cicli)
- 3) eliminare i simboli inutili

allora la grammatica risultante è garantita non avere né ϵ -produzioni, né produzioni unitarie, né simboli inutili, ed è equivalente a quella di partenza!

N.B. L'ordine è importante poiché alcune delle costruzioni possono interagire tra loro!

Ad es.: durante la fase di eliminazione delle ϵ -produzioni, potremmo introdurre prod. unitarie!

$\Rightarrow$ le ϵ -produzioni vanno eliminate prima di procedere all'eliminazione delle produzioni unitarie

$$\left. \begin{array}{l} S \rightarrow a A a \\ A \rightarrow C \\ C \rightarrow S \mid \varepsilon \end{array} \right\} G$$

1) Togliere ε -production

$$N(G) = \{C, A\}$$

 $\Downarrow$

$$G' \left[\begin{array}{l} S \rightarrow a A a \mid aa \\ A \rightarrow C \\ C \rightarrow S \end{array} \right.$$

- tolgo $C \rightarrow \varepsilon$
 - non metto
 $A \rightarrow \varepsilon$

2) Togliere le prod. unitarie

$$U(G') = \{(A, A), (C, C), (S, S), (A, C), (C, S), (A, S)\}$$

$$G'' \left[\begin{array}{l} S \rightarrow a A a \mid aa \\ C \rightarrow a A a \mid aa \\ A \rightarrow a A a \mid aa \end{array} \right.$$

perché $(S, S) \in U(G')$ perché $(C, S) \in U(G')$ perché $(A, S) \in U(G')$ 3) Rimuovere i simboli inutili

$$G(G'') = \{S, a, A, C\} \text{ tutti generatori}$$

$$R(G'') = \{S, a, A\} \text{ ma non } C!$$

 $\Downarrow$

$$\left[\begin{array}{l} S \rightarrow a A a \mid aa \\ A \rightarrow a A a \mid aa \end{array} \right] G'''$$

È possibile trovare una grammatica ancora più semplice se $L(G''') = (aa)^+$?

$$S \rightarrow a S a \mid aa$$

o anche

$$S \rightarrow aaS \mid aa$$

Forme Normali

(34)

- di Chomsky
- di Greibach

garantiscono proprietà particolari alle grammatiche

Forma normale di Chomsky

Tutte le sue produzioni sono della forma

$$A \rightarrow BC$$

$$A \rightarrow a$$

(con ϵ trattato a parte:

$$S \rightarrow \epsilon \mid BC \dots)$$

Oss: se G libera è in forma normale di Chomsky, allora

- non ha ϵ -produzioni
- non ha produzioni unitarie

(S non compare mai a dx in una produzione)

Oss: Ogni grammatica libera G può essere trasformata in una equivalente G' in forma normale di Chomsky.

Forma Normale di Greibach

Tutte le produzioni sono della forma

$$A \rightarrow aBC$$

$$A \rightarrow aB$$

$$A \rightarrow a$$

(con ϵ trattato a parte, come sopra)

Oss: - no ϵ -produzioni / no prod. unitarie

- non è mai ricorsiva sx!

- ogni prod., applicata in una derivazione, allunga il prefisso di terminali $\Rightarrow$ parser costruito a partire da f.n. di Greibach sono meno nodati.

Oss: Ogni G libera può essere trasformata in una equiv. f.n. Greibach

Eliminare la ricorsione sinistra (35)

(problema per parser
top-down)

- produzione ricorsiva sx : $A \rightarrow A\alpha \in R$
- G è ricorsiva sx : $A \Rightarrow^+ A\alpha$ per qualche $\alpha \in (T \cup N)^*$

Come rimuovere la ricorsione sx immediata?

$$A \rightarrow A\alpha_1 | \dots | A\alpha_n | \beta_1 | \dots | \beta_m$$

(le stringhe β_i non cominciano per A)

Queste produzioni possono essere rimpiattate da

$$A \rightarrow \beta_1 A' | \dots | \beta_m A'$$

$$A' \rightarrow \alpha_1 A' | \dots | \alpha_n A' | \epsilon$$

Se nella gr. originale abbiamo la derivazione

$$A \Rightarrow A\alpha_{i_1} \Rightarrow A\alpha_{i_2}\alpha_{i_1} \Rightarrow \dots \Rightarrow A\alpha_{i_k}\dots\alpha_{i_2}\alpha_{i_1} \Rightarrow \beta_i\alpha_{i_k}\dots\alpha_{i_2}\alpha_{i_1}$$

allora nella nuova grammatica avremo

$$\begin{aligned} A \Rightarrow \beta_i A' &\Rightarrow \beta_i \alpha_{i_k} A' \Rightarrow \dots \Rightarrow \beta_i \alpha_{i_k} \dots \alpha_{i_2} A' \Rightarrow \beta_i \alpha_{i_k} \dots \alpha_{i_2} \alpha_{i_1} A' \\ &\Rightarrow \beta_i \alpha_{i_k} \dots \alpha_{i_2} \alpha_{i_1} \end{aligned}$$

e viceversa

Esempio

(36)

$$A \rightarrow Aa \mid b$$



$$A \rightarrow bA'$$

$$A' \rightarrow aA' \mid \varepsilon$$

Esempio

$$A \rightarrow Ab \mid Ac \mid d$$



$$A \rightarrow dA'$$

$$A' \rightarrow bA' \mid cA' \mid \varepsilon$$

Oss: Se $G = [A \rightarrow Aa]$, non si può

applicar l'algoritmo, perché mancano le produzioni di base da cui partire ($A \rightarrow \beta_1 \mid \dots \mid \beta_m$)

Infatti, $L(G) = \emptyset$ e la grammatica corrispondente non ha produzioni.

Ricorsione sx non-immediata

(37)

Consideriamo

$$\left. \begin{array}{l} S \rightarrow Ba \mid b \\ B \rightarrow Bc \mid Sc \mid d \end{array} \right\} G$$

In G c'è ricorsione sx immediata ($B \rightarrow Bc$),
ma anche non-immediata ($S \Rightarrow Ba \Rightarrow Sca$).

Algoritmo: Input: G libera senza ϵ -produzioni, senza
prod. unitarie, ma con ricorsione sx
non-immediata

Output: G' libera, senza ricorsione sx, ma
può avere ϵ -produzioni

Sia $NT = \{A_1, A_2, \dots, A_n\}$ in un ordine fissato

For i from 1 to n {

1) For j from 1 to $i-1$ {

sostituisci ogni produzione della forma

$A_i \rightarrow A_j \alpha$ con le produzioni

$A_i \rightarrow \beta_1 \alpha \mid \dots \mid \beta_k \alpha$ dove $A_j \rightarrow \beta_1 \mid \dots \mid \beta_k$
sono le produzioni contenute per A_j

}

2) Elimina la ricorsione immediata su A_i

}

- L'obiettivo dell'algoritmo è che, alla fine, ogni produzione del tipo $A_i \rightarrow A_K \alpha$ sia tale che $i < K$, in modo che sia impossibile avere ricorsione Sx non immediata.
- Quando $i=1$, l'unica cosa che viene fatta è l'istruzione 2), che rimuove l'eventuale ricorsione Sx immediata. Al Termine, se $A_1 \rightarrow A_K \alpha$ avremo $1 < K$.
- Alla i -esima iterazione del for esterno, tutti i nonterminali A_m con $m < i$ hanno produzioni con la proprietà desiderata.
Ora il ciclo for interno (istruzione 1) aumenta progressivamente l'indice del nonterminale in prima posizione, finché, al termine del ciclo ($j=i-1$), avremo che ogni produzione $A_i \rightarrow A_K \alpha$ è tale che $i \leq K$.
Ora l'istruzione 2) rimuove l'eventuale ricorsione Sx immediata da A_i , sicché ogni produzione $A_i \rightarrow A_K \alpha$ è tale che $i < K$.
- Quindi al Termine dell'algoritmo⁽ⁱ⁼ⁿ⁾, avremo che ogni produzione $A_i \rightarrow A_K \alpha$ è tale che $i < K$, garantendo l'impossibilità di creare ricorsione Sx non immediata.

$$S \rightarrow Ba | b$$

$$B \rightarrow Bc | Sc | d$$

$$NT = \begin{matrix} \{S, B\} \\ 1 \quad 2 \end{matrix}$$

(38)

- Per $i=1$ (cioè per S), il ciclo interno (J da 1 a $\emptyset$) non viene eseguito; siccome non c'è ricorsione immediata per S , non faccio niente!
- Per $i=2$ (cioè $A_i = B$), il ciclo interno (J da 1 a 1) si esegue solo per $A_J = A_1 = S$.
Allora la produzione $B \rightarrow Sc$ viene rimpiazzata con

$$B \rightarrow Bac | bc$$

Ora le produzioni complessive per B sono

$$B \rightarrow Bc | Bac | bc | d$$

dalle quali dobbiamo eliminare la ricorsione immediata!

Il risultato è

$$B \rightarrow bcB' | dB'$$

$$B' \rightarrow cB' | acB' | \epsilon$$

ovvero la grammatica risultante è

$$S \rightarrow Ba | b$$

$$B \rightarrow bcB' | dB'$$

$$B' \rightarrow cB' | acB' | \epsilon$$

Perché si richiede che la G in input (38 bis)
non abbia produzioni unitarie e produzioni epsilon?

$$\left. \begin{array}{l} S \rightarrow AB | a \\ A \rightarrow \epsilon \\ B \rightarrow S | b \end{array} \right\}$$

OSS! $S \Rightarrow AB \Rightarrow B \Rightarrow S$

$$S \Rightarrow^+ S$$

(quindi c'è un ciclo
zic. sx. non immediato)

$S=1 \quad A=2 \quad B=3$

Per $i=1, j=1$ a 0 non viene fatto niente

Per $i=2, j=1$ ancora niente

$$S \rightarrow AB | a$$
$$A \rightarrow \epsilon$$
$$B \rightarrow S | b$$

ma per $i=3$ e $j=1$

$$S \rightarrow AB | a$$
$$A \rightarrow \epsilon$$
$$B \rightarrow AB | a | b$$

e per $i=3$ e $j=2$

$$S \rightarrow AB | a$$
$$A \rightarrow \epsilon$$
$$B \rightarrow B | a | b$$

e compare una prod.

zic. sx $B \rightarrow B$

del tutto insensata
perché non serve a
niente

$$\left. \begin{array}{l} S \rightarrow AB \mid D \\ A \rightarrow \varepsilon \mid Ab \mid aB \\ B \rightarrow b \mid bB \\ C \rightarrow c \\ D \rightarrow aD \end{array} \right\} G$$

- 1) Toplene simbolo inutile
- 2) Toplene ricorsione sx
- 3) Toplene ε -production
- 4) Toplene prod. unitaria

$$\begin{array}{l} 1) D \text{ non generatore} \\ C \text{ non raggiungibile} \end{array} \Rightarrow \left. \begin{array}{l} S \rightarrow AB \\ A \rightarrow \varepsilon \mid Ab \mid aB \\ B \rightarrow b \mid bB \end{array} \right\} G'$$

$$\begin{array}{l} 2) \text{ Ricorsione sx} \\ (A \rightarrow Ab) \end{array} \Rightarrow \left. \begin{array}{l} S \rightarrow AB \\ A \rightarrow A' \mid aBA' \\ A' \rightarrow bA' \mid \varepsilon \\ B \rightarrow b \mid bB \end{array} \right\} G''$$

$$\begin{array}{l} 3) \text{ Toplene le } \varepsilon\text{-prod.} \\ N(G'') = \{A, A'\} \end{array} \Rightarrow \left. \begin{array}{l} S \rightarrow AB \mid B \\ A \rightarrow A' \mid aBA' \mid aB \\ A' \rightarrow bA' \mid b \\ B \rightarrow b \mid bB \end{array} \right\} G'''$$

4) Toplene le prod. unitarie

$$U(G''') = \text{Id} U \{ (S, B), (A, A') \}$$

$\Rightarrow$

$$\left. \begin{array}{l} S \rightarrow AB \mid \underline{b \mid bB} \\ A \rightarrow \underline{bA' \mid b} \mid aBA' \mid aB \\ A' \rightarrow bA' \mid b \\ B \rightarrow b \mid bB \end{array} \right\}$$

Fattorizzazione a Sinistra

(40)

(importante per top-down parsing)

$$A \rightarrow aBbC \mid aBd$$

Se, in top-down parsing, sulla pila ha A e leggo in input a , non sono in grado di determinare quale produzione scegliere (nondeterminismo)

$\Rightarrow$ raccolgo la parte comune (" aB ") alle 2 produzioni e introduco un nuovo nonterminale per rappresentare il "resto"

$$A \rightarrow aBA'$$

$$A' \rightarrow bC \mid d$$

Algoritmo generale

- Inizializza $N = NT$
 - Ripeti il ciclo presente finché nessuna modifica è più possibile a N o all'insieme delle produzioni {
for each $A \in N$ {
Sia α il prefisso più lungo comune alle parti destre di alcune produzioni di A ;
if $\alpha \neq \epsilon$ {
 - sia A' un nuovo nonterminale, $N := N \cup \{A'\}$;
 - rimpiazza tutte le produzioni per A
 $A \rightarrow \alpha\beta_1 \mid \dots \mid \alpha\beta_k \mid \gamma_1 \mid \dots \mid \gamma_r$
con le produzioni
 $A \rightarrow \alpha A' \mid \gamma_1 \mid \dots \mid \gamma_r$
 $A' \rightarrow \beta_1 \mid \dots \mid \beta_k$}
- }

Esempio

(41)

$$E \rightarrow T \mid T + E \mid T - E$$

$$T \rightarrow A \mid A * T$$

$$A \rightarrow a \mid b \mid (E)$$

} simpler
factorizations!

$$E \rightarrow T E'$$

$$E' \rightarrow \varepsilon \mid + E \mid - E$$

$$T \rightarrow A T'$$

$$T' \rightarrow \varepsilon \mid * T$$

$$A \rightarrow a \mid b \mid (E)$$

} Versions
equivalent
factorizations